

Coefficient-Guided Model Trees for Piecewise Linear Regression: From a Recursive Baseline to Adaptive Feature Screening

[Author(s) — Affiliation(s)]

Abstract

We present a unified treatment of coefficient-guided model trees for piecewise linear regression, tracing the development from a minimal baseline to a robust adaptive variant. We first introduce the Recursive Hybrid Model (RHM), a Linear Model Tree that recursively partitions the feature space using binary splits and fits a local Ridge regression model at each leaf. RHM’s distinguishing design choice is a Ridge-regression-guided feature selection step: at each internal node, RHM fits a Ridge regression and selects the feature with the largest absolute coefficient magnitude as the splitting axis—the *top-1 heuristic*. This reduces per-node feature selection from an exhaustive $O(nd^2 \cdot d)$ cost to a single Ridge fit, while remaining data-driven. We provide a precise algorithmic description of RHM, derive its training complexity $O(D \cdot n \cdot d^2)$ and inference complexity $O(m(D + d))$, and establish analytically that the piecewise linear approximation converges at the classical $O(h^2)$ rate with increasing tree depth.

We then identify the principal weakness of the top-1 heuristic: coefficient magnitudes can be distorted by feature scaling, collinearity, and regime-dependent structure, causing RHM to commit to a suboptimal splitting axis. To address this, we introduce the Adaptive Coefficient-Guided Model Tree (ACGMT), whose central enhancement is *top- k feature preselection*: instead of committing to a single coefficient-ranked feature, ACGMT screens the top- k candidates and evaluates each for the best split. We additionally study node-wise standardization, adaptive coarse-to-fine threshold search, and validation-aware pruning. Ablation experiments on five synthetic stress tests (scaling, correlated predictors, irrelevant features, interaction structure, and piecewise regimes) show that top- k screening is the dominant improvement: test MSE drops from 4.746 to 0.385 on the interaction dataset and from 2.791 to 0.352 on the regime dataset when moving from top-1 to top-3. Adaptive grid refinement and pruning provide supplementary, context-dependent gains; standardization is neutral to harmful. ACGMT remains a single interpretable tree and fits faster than gradient boosting in all evaluated settings.

1 Introduction

Many real-world regression problems exhibit regime-dependent behaviour: the relationship between predictors and response is approximately linear within local regions of the feature space but differs substantially across those regions. Globally linear models such as Ridge or Lasso regression cannot capture these local variations, while fully non-parametric methods such as Gaussian Processes may be computationally prohibitive at scale.

Decision-tree-based methods handle partitioning naturally, but classical regression trees (CART; Breiman et al. 1984) assign a constant value at each leaf—a poor approximation when the within-region signal is itself linear. The class of Linear Model Trees (also called Model Trees) addresses this by fitting linear models at the leaves of a decision tree. This idea has been explored since at least Friedman [1979], popularised by Quinlan’s M5 [Quinlan, 1992] and its smoothed successor M5’ [Wang & Witten, 1997], and continues to receive active attention in recent work such as PILOT [Raymaekers et al., 2024] and PINE [Li et al., 2024].

This paper makes two contributions within this family. The first is the Recursive Hybrid Model (RHM), a minimal and interpretable Linear Model Tree whose primary contribution is a specific, underexplored feature selection mechanism: at each internal node, a Ridge regression is fitted to the current data partition, and the feature with the largest absolute coefficient magnitude is chosen as the splitting axis. This replaces the exhaustive feature search used in CART and M5, offering a computationally cheaper alternative that is still guided by the data. The split point along the chosen axis is then selected by scanning a percentile grid and minimising a weighted MSE of child Ridge fits. We provide a formal algorithmic description, time and space complexity bounds, and an analytical characterisation of its $O(h^2)$ convergence in idealised settings.

The second contribution addresses the principal weakness of this top-1 heuristic. While computationally attractive, committing to a single coefficient-ranked feature at each node is fragile: coefficient magnitudes are distorted by feature scaling, correlated predictors spread signal across multiple variables, and regime-dependent structure may require exploring more than one candidate axis. We therefore introduce the Adaptive Coefficient-Guided Model Tree (ACGMT), which keeps the overall RHM structure but replaces the top-1 split commitment with a *top-k feature screening* step: the algorithm ranks features by Ridge coefficient magnitude, retains the k highest-ranked candidates, and evaluates each for the best split. This single change is the paper’s central practical contribution. We additionally study three supplementary refinements—node-wise standardization, adaptive coarse-to-fine threshold search, and validation-aware pruning—and establish empirically which of these help and in what contexts.

The paper is organised as follows. Section 2 reviews the Linear Model Tree literature and positions both RHM and ACGMT within it. Section 3 gives the full RHM algorithm with complexity and convergence analysis. Section 4 analyses time and space complexity. Section 5 identifies RHM’s limitations and presents ACGMT. Section 6 describes the experimental setup and reports results for both RHM and ACGMT. Section 7 discusses findings and limitations. Section 8 concludes.

2 Related Work

Both RHM and ACGMT belong to the Linear Model Tree (LMT) family. We review the main prior algorithms and clarify where each contribution sits within this space.

CART [Breiman et al., 1984]. The foundational regression tree assigns a constant (mean of targets) at each leaf and selects splits by exhaustive search over all features and thresholds to minimise within-node MSE. RHM replaces the constant leaf with a Ridge model and the exhaustive feature search with a coefficient-guided heuristic.

M5 and M5’ [Quinlan, 1992, Wang & Witten, 1997]. M5 is the canonical Model Tree algorithm. It grows the tree structure using a standard variance-reduction criterion and then, once the structure is fixed, re-fits linear models at the leaves and applies M5’-style smoothing across node boundaries to improve continuity. RHM differs in three ways: (1) Ridge regression is used rather than OLS, providing L_2 regularisation at the leaf level; (2) feature selection uses coefficient magnitudes rather than variance reduction; (3) no post-hoc smoothing is applied. RHM is therefore simpler but less polished than M5’.

PILOT [Raymaekers et al., 2024]. PILOT is a recent, theoretically grounded LMT with the same asymptotic complexity as CART. It selects features using an L_2 -boosting approach—fitting a univariate piecewise linear model and passing residuals to child nodes—rather than fitting a full multivariate Ridge at each node. PILOT also applies truncation to mitigate extrapolation. Relative to PILOT, RHM fits a full multivariate Ridge for feature selection (more expensive per

node but uses all features jointly) and fits independent Ridge models at each child rather than boosting residuals.

PINE [Li et al., 2024]. PINE is a performance-oriented LMT that uses Ridge regression at both internal nodes and leaves, GPU-accelerated histogram construction, and prediction bounding. PINE is the closest existing algorithm to RHM in terms of the leaf estimator (Ridge) and the use of Ridge at internal nodes for splitting. The primary distinction is that PINE evaluates split quality via a full MSE search over all features and binned thresholds, whereas RHM uses coefficient magnitude to pre-select a single feature and then scans a coarser percentile grid.

linear-tree [Cerlymarco, 2021]. An open-source Python package providing scikit-learn-compatible LMT implementations with pluggable linear estimators (including Ridge). It evaluates splits by fitting the chosen linear model on both child partitions and computing a criterion score, similar in spirit to RHM’s split search but with an exhaustive feature search. RHM’s coefficient-guided feature selection is the key difference.

Piecewise Linear Regression and MARS. Friedman’s MARS [Friedman, 1991] constructs a piecewise linear function using spline basis functions chosen greedily, without an explicit tree structure. Threshold regression [Hansen, 2000] estimates break-points in econometric linear models. These methods share the piecewise-linear spirit but differ structurally from tree-based partitioning.

Random Forests and Gradient Boosting. Random forests [Breiman, 2001] and gradient boosting regression trees [Friedman, 2001] provide strong non-linear baselines by aggregating many trees, though at the cost of larger ensembles and lower local interpretability. Our goal is not to replace such methods across the board, but to identify the regime where a single adaptive piecewise linear tree remains competitive and to understand the failure modes of coefficient-guided splitting.

Summary of positioning. RHM is a minimal, interpretable LMT distinguished by its coefficient-guided feature selection heuristic. ACGMT is a direct continuation of the same idea that makes the search substantially more robust. Neither method is claimed to outperform PILOT, PINE, or tree ensembles on general benchmarks; rather, they offer a transparent, analysable design that illuminates the trade-offs between exhaustive feature search and coefficient-proxy selection strategies.

3 The Recursive Hybrid Model (RHM)

We now describe the RHM algorithm in full detail, covering data structures, tree construction, split optimisation, inference, structural properties, and convergence analysis.

3.1 Data Structures

The model is represented as a binary tree of `HybridNode` objects. Each node stores:

- **feature**—the name of the splitting feature (`None` for leaf nodes);
- **split_point**—the threshold value τ such that samples with **feature** $\leq \tau$ route left (`None` for leaf nodes);
- **left, right**—child `HybridNode` references (`None` for leaf nodes);

- **model**—a fitted Ridge regression object (**None** for internal nodes).

A node is a leaf if and only if its **model** attribute is not **None**. Internal nodes have no model and perform routing only.

3.2 Tree Construction: $\text{fit}(X, y)$

The public interface follows the scikit-learn estimator convention: $\text{fit}(X, y)$ stores the fitted tree in **self.root** and returns **self**. Internally, it delegates to a recursive procedure $\text{_build_tree}(X, y, \text{depth})$ (Algorithm 1) which constructs the tree top-down starting at **depth** = 0.

Algorithm 1: $\text{RHM._build_tree}(X, y, \text{depth})$

Input: $X \in \mathbb{R}^{n \times d}$, $y \in \mathbb{R}^n$, $\text{depth} \in \mathbb{N}$ **Output:** **HybridNode** (root of subtree)

1. **if** $\text{depth} \geq \text{max_depth}$ **or** $n < \text{min_samples_split}$ **then**
2. $m \leftarrow \text{Ridge}(\alpha=1.0).\text{fit}(X, y)$
3. **return** **HybridNode**($\text{model}=m$) ▷ leaf
- // Step A: feature selection via Ridge coefficients*
4. $m_{\text{node}} \leftarrow \text{Ridge}(\alpha=1.0).\text{fit}(X, y)$
5. $j^* \leftarrow \arg \max_j |m_{\text{node}}.\text{coef_}[j]|$ ▷ most significant feature
- // Step B: split-point search over percentile grid*
6. $P \leftarrow \{p_{10}, p_{20}, \dots, p_{90}\}$ of $X[:, j^*]$ ▷ 9 candidates
7. $\text{best_mse} \leftarrow \infty$; $\text{best_}\tau \leftarrow \text{None}$
8. **for each** $\tau \in P$ **do**
9. $L \leftarrow \{i : X[i, j^*] \leq \tau\}$; $R \leftarrow \text{complement}$
10. **if** $|L| < \text{min_samples_split}$ **or** $|R| < \text{min_samples_split}$: **continue**
11. $m_L \leftarrow \text{Ridge}(\alpha=1.0).\text{fit}(X[L], y[L])$
12. $m_R \leftarrow \text{Ridge}(\alpha=1.0).\text{fit}(X[R], y[R])$
13. $\text{mse} \leftarrow (|L| \cdot \text{MSE}(y[L], m_L(X[L])) + |R| \cdot \text{MSE}(y[R], m_R(X[R]))) / n$
14. **if** $\text{mse} < \text{best_mse}$: $\text{best_mse} \leftarrow \text{mse}$; $\text{best_}\tau \leftarrow \tau$
15. **if** $\text{best_}\tau = \text{None}$: **return** **HybridNode**($\text{model}=m_{\text{node}}$) ▷ fallback leaf
- // Step C: partition and recurse*
16. $L^* \leftarrow \{i : X[i, j^*] \leq \text{best_}\tau\}$; $R^* \leftarrow \text{complement}$
17. $\text{left} \leftarrow \text{_build_tree}(X[L^*], y[L^*], \text{depth} + 1)$
18. $\text{right} \leftarrow \text{_build_tree}(X[R^*], y[R^*], \text{depth} + 1)$
19. **return** **HybridNode**($\text{feature}=j^*$, $\text{split_point}=\text{best_}\tau$, $\text{left}=\text{left}$, $\text{right}=\text{right}$)

3.3 Step A: Feature Selection via Ridge Coefficients

Rather than evaluating all d features as splitting candidates (as CART does, with $O(nd \log n)$ cost per level), RHM first fits a Ridge regression to the current partition to obtain a global linear signal. The feature with the largest absolute coefficient magnitude, $j^* = \arg \max_j |\beta_j|$, is selected as the splitting feature. This heuristic is motivated by the assumption that the feature that dominates the global linear signal within the current region is the most informative axis along which to further partition the space.

A key advantage is computational efficiency: feature selection reduces to a single Ridge fit plus an argmax, costing $O(nd^2)$ rather than $O(nd^2 \times d)$. The Ridge regularisation ($\alpha = 1.0$) also guards against coefficient inflation under multicollinearity, making the selection more stable than with OLS.

3.4 Step B: Split-Point Optimisation

Given splitting feature j^* , the optimal split threshold τ^* is found by scanning a fixed grid of 9 candidate values—the 10th through 90th percentiles of $X[:, j^*]$ in steps of 10 (lines 10–21 of Algorithm 1). For each candidate τ :

1. The node samples are partitioned into a left set $L = \{i : X[i, j^*] \leq \tau\}$ and right set $R = \{i : X[i, j^*] > \tau\}$.
2. A Ridge model is fitted independently on each partition.
3. The weighted MSE criterion is computed:

$$\text{MSE}(\tau) = \frac{|L| \cdot \text{MSE}_L + |R| \cdot \text{MSE}_R}{n}$$

where MSE_L and MSE_R are the mean squared errors of the fitted models on their respective partitions.

This criterion is a sample-weighted average of the two residual variances and is analogous to the impurity-reduction criterion in CART but evaluated on Ridge-fitted residuals rather than pure variance.

A split is admissible only if both L and R have at least `min_samples_split` samples. If no admissible split is found for any τ in the grid, the current node becomes a leaf.

The percentile grid avoids extreme splits (below p_{10} or above p_{90}) that would create highly imbalanced partitions. Using 9 fixed percentile thresholds rather than all unique values gives a significant speed-up while retaining good granularity in practice.

3.5 Step C: Recursion and Termination

Once the best split (j^*, τ^*) is determined, the algorithm recurses on the two child partitions at depth + 1. Recursion terminates when either:

- (a) $\text{depth} \geq \text{max_depth}$: the maximum allowable tree depth is reached, or
- (b) $n < \text{min_samples_split}$: the partition contains fewer samples than the minimum required to attempt a split.

Both conditions create a leaf node: a Ridge model is fitted on the local partition and stored in the node. During inference, all samples that reach this leaf will be predicted by this local model.

3.6 Inference: `predict(X)`

Algorithm 2 describes prediction for a new sample $x \in \mathbb{R}^d$.

Algorithm 2: `RHM.predict(x)`
Input: $x \in \mathbb{R}^d$ (single test sample) **Output:** $\hat{y} \in \mathbb{R}$

1. `node` $\leftarrow$ `root`
2. **while** `node.model` is `None` **do**
3. **if** $x[\text{node.feature}] \leq \text{node.split_point}$ **then**
4. `node` $\leftarrow$ `node.left`
5. **else**
6. `node` $\leftarrow$ `node.right`
7. **return** `node.model.predict(x)`

For a dataset of m test samples, Algorithm 3.6 is applied independently to each row. The prediction is fully deterministic and interpretable: one can trace the exact sequence of binary decisions that led to the leaf model used.

3.7 Structural Properties

Piecewise linearity. The function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ computed by RHM is piecewise linear. The feature space is partitioned into at most $2^{\max_depth}$ axis-aligned hyper-rectangular regions, and on each region f is an affine function (the leaf Ridge model).

Continuity. RHM does not enforce continuity at region boundaries. Adjacent leaf models may predict different values for the same input placed exactly at a split threshold—a standard characteristic of hard-routing trees. In the special case of a purely quadratic target with symmetric midpoint splits and population-level Ridge fits, adjacent leaf models happen to agree exactly at their shared boundary (see the analytical examples in Section 3.8). On finite samples with regularisation and percentile-grid splits, small discontinuities will generally arise.

Interpretability. Each leaf is associated with a human-readable decision path (a conjunction of threshold comparisons) and a set of $d + 1$ interpretable coefficients from its Ridge model.

3.8 Numerical Example

To build intuition, we analyse the RHM algorithm on idealised problems where the population-level Ridge fits admit closed-form solutions. The coefficients and error bounds below are *analytical* (computed from the continuous data distribution, not from finite samples); the implemented algorithm approximates these values on sampled data with regularisation.

Setup. Consider x drawn uniformly from $[-3, 3]$ with a single feature ($d = 1$) and target

$$y = 2x + x^2.$$

The true function is a parabola—globally non-linear, but locally well-approximated by straight lines. We apply RHM with $\max_depth = 1$.

Step A: Feature selection. With $d = 1$ feature, the Ridge fit at the root returns a single coefficient $\hat{\beta}_1$. Since there is only one candidate, $j^* = x$ is selected automatically. The population-level Ridge fit (dashed line in Figure 1) is $\hat{y} = 2x + 3$ (slope equals the linear coefficient; intercept equals $\mathbb{E}[x^2] = 3$). This captures the linear trend but systematically misses the curvature.

Step B: Split-point search. Scanning the percentile grid, the weighted MSE is minimised at $\tau^* = 0.0$ (the median), which splits the parabola at its vertex—the point of maximum curvature.

Step C: Leaf models. Two Ridge models are fitted (using the closed-form population-level coefficients derived in the convergence analysis below):

- **Left leaf** ($x \leq 0$): $\hat{y}_L = -x - 1.5$. On the left half, the negative slope of the linear term and the positive curvature of x^2 partially cancel, producing a negative slope.
- **Right leaf** ($x > 0$): $\hat{y}_R = 5x - 1.5$. On the right half, both terms are increasing, yielding a steep positive slope.

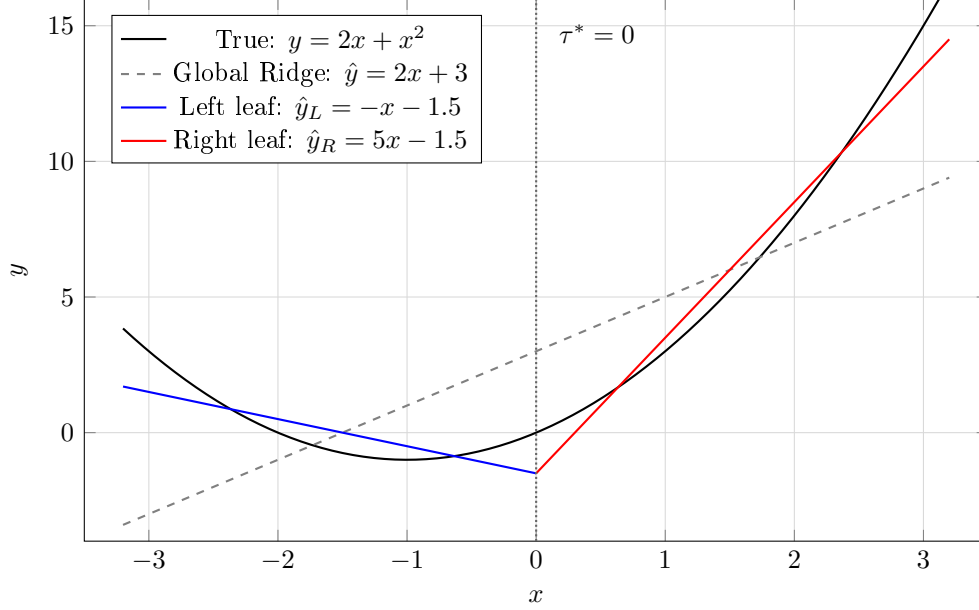


Figure 1: Analytical example with $d = 1$ and $y = 2x + x^2$. The global Ridge fit (dashed grey) captures only the average linear trend. RHM with $\text{max_depth} = 1$ splits at $\tau^* = 0$ and fits population-level Ridge models on each half (blue, red), reducing the maximum pointwise error by $4\times$ relative to the global fit. All coefficients are closed-form population values.

Result. The piecewise linear approximation (solid lines in Figure 1) tracks the parabola much more closely than the global Ridge fit. The maximum pointwise error drops from 6.0 (global Ridge at the endpoints) to 1.5 (RHM, depth 1)—a $4\times$ reduction—because each leaf adapts its slope to the local regime.

This example illustrates the core mechanism: RHM identifies the axis of greatest variation (here the only feature), finds the split point that best separates distinct linear regimes, and fits local models that together approximate the non-linear surface far more accurately than any single linear model.

3.9 Convergence with Increasing Depth

The depth-1 example above shows a single split reducing MSE by 87%. A natural question is: how does the approximation improve as we continue splitting? Figure 2 answers this by showing the piecewise linear approximation of $y = 2x + x^2$ on $[-3, 3]$ at depths 1 through 4, producing 2, 4, 8, and 16 leaf segments respectively.

On each segment $[a, b]$ of width $h = b - a$, the population-level Ridge fit has a closed-form solution: slope $\hat{\beta} = 2 + a + b$ (the average derivative of $2x + x^2$ on $[a, b]$) and intercept $\hat{\beta}_0 = -(a^2 + 4ab + b^2)/6$. A notable property of this idealised setting emerges: *adjacent leaf models agree exactly at their shared boundary*. That is, for any two neighbouring segments $[a, m]$ and $[m, b]$ with $m = (a + b)/2$, the left and right Ridge fits predict the same value at $x = m$. The piecewise approximation is therefore continuous—a polygon that wraps smoothly around the parabola with no gaps at the split points. (On finite samples, small discontinuities will generally be present; see Section 3, Structural Properties.)

The maximum approximation error on each segment is $h^2/6$, where h is the segment width. Since each depth level halves h , the error decreases by a factor of four per level (Table 1). This is the classical $O(h^2)$ convergence rate of piecewise linear approximation applied to smooth (C^2) functions.

The continuity at boundaries is a consequence of the symmetry of the quadratic: when a

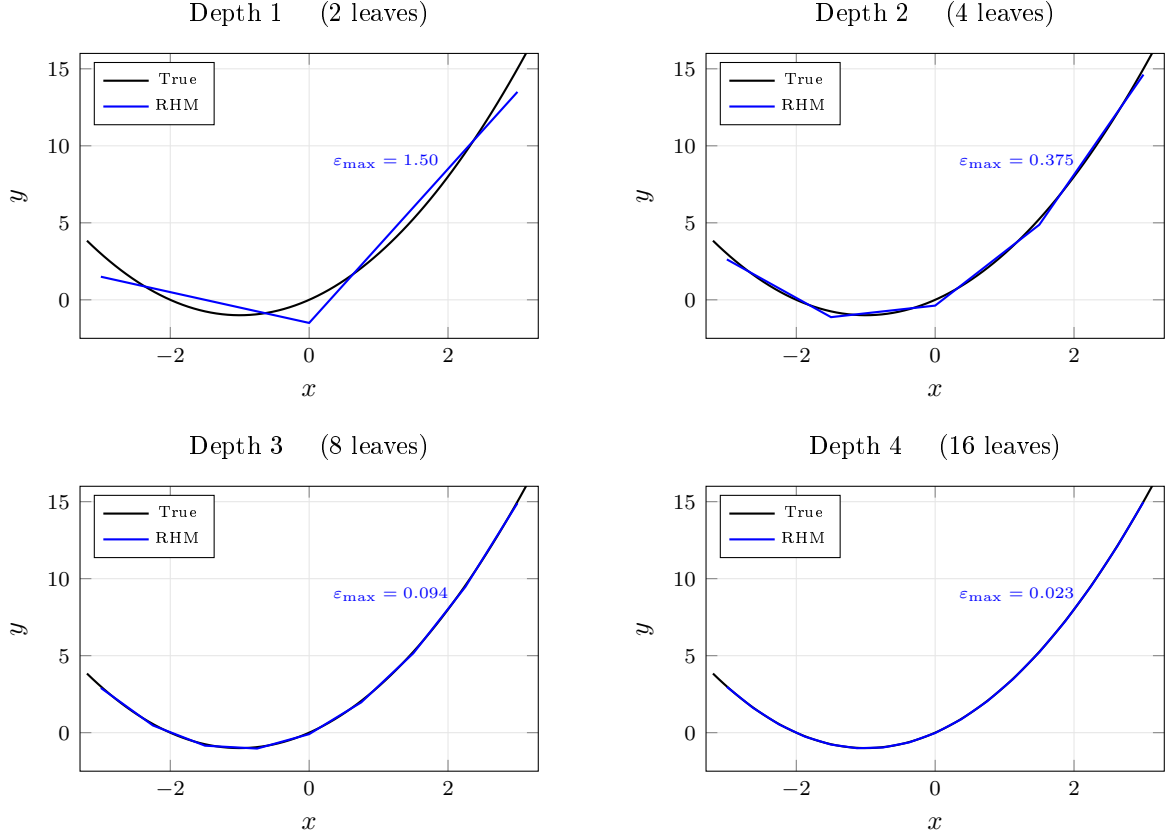


Figure 2: Piecewise linear approximation of $y = 2x + x^2$ at increasing tree depth. At each depth k , the interval $[-3, 3]$ is partitioned into 2^k segments. The approximation (blue polyline) wraps progressively tighter around the true parabola (black curve), with no discontinuities at segment boundaries. The maximum error $\epsilon_{\max}$ decreases by a factor of 4 with each depth level.

segment is split at its midpoint, the two child Ridge fits yield identical predictions at the split point. This property holds exactly for quadratic targets and approximately for smooth functions with moderate higher-order derivatives.

3.10 Extension to Two Dimensions

For the separable function $y = 3x_1 - 2x_2 + x_1^2 + x_2^2$, the same convergence applies independently along each axis. Figure 3 shows the partition of $[-2, 2]^2$ produced by RHM at depth 2. The algorithm adapts its splitting axis at each node based on the Ridge coefficients: the root splits on x_1 (since $|\hat{\beta}_1| = 3 > |\hat{\beta}_2| = 2$); the left child, where the quadratic x_1^2 has reduced $|\hat{\beta}_1|$ to 1, switches to splitting on x_2 ($|\hat{\beta}_2| = 2$ now dominates); the right child, where x_1^2 has amplified $|\hat{\beta}_1|$ to 5, continues splitting on x_1 .

With increasing depth, the tree continues to subdivide each region along the axis with the largest Ridge coefficient, producing smaller rectangular cells whose Ridge planes approximate the paraboloid with $O(h^2)$ error in each axis.

Surface approximation with 32 leaf planes. Figure 4 demonstrates this convergence in two dimensions. We partition $[-2, 2]^2$ into an 8×4 grid of 32 rectangular cells—equivalent to a depth-5 tree with alternating axis splits (5 levels produce $2^5 = 32$ leaves, distributed as 8 strips in x_1 and 4 strips in x_2). On each cell $[a_1, b_1] \times [a_2, b_2]$, the Ridge fit has closed-form coefficients:

$$\hat{\beta}_1 = 3 + a_1 + b_1, \quad \hat{\beta}_2 = -2 + a_2 + b_2, \quad \hat{\beta}_0 = -\frac{a_1^2 + 4a_1b_1 + b_1^2}{6} - \frac{a_2^2 + 4a_2b_2 + b_2^2}{6}.$$

Table 1: Convergence of the piecewise linear approximation with tree depth.

Depth	Leaves	Segment width h	Max error $h^2/6$	Reduction
1	2	3.000	1.500	—
2	4	1.500	0.375	4×
3	8	0.750	0.094	4×
4	16	0.375	0.023	4×

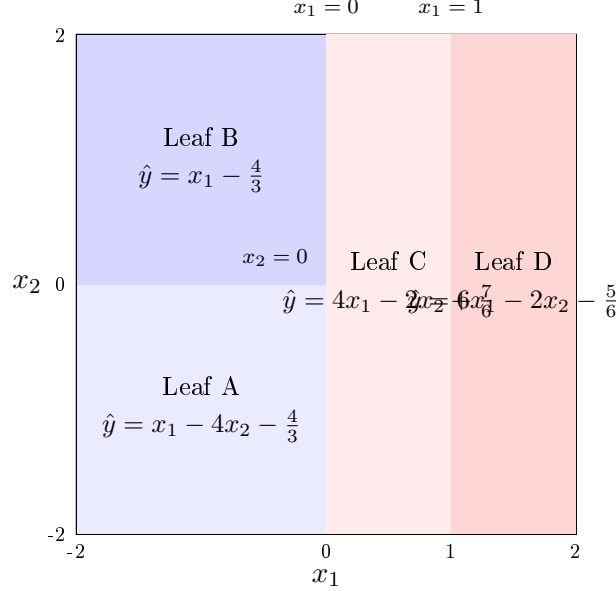


Figure 3: Partition of $[-2, 2]^2$ by RHM at depth 2 for $y = 3x_1 - 2x_2 + x_1^2 + x_2^2$. The root splits on x_1 at 0. The left child splits on x_2 at 0 (where x_2 dominates after the x_1^2 curvature reduces $|\hat{\beta}_1|$). The right child splits on x_1 at 1 (where x_1 still dominates). Each leaf shows its fitted Ridge plane. The adaptive axis selection at each node is a distinguishing feature of the coefficient-guided heuristic.

Because the target is separable, adjacent cells produce Ridge planes that *agree exactly at their shared boundary*—the 32-plane surface is fully continuous. The maximum error is $h_1^2/6 + h_2^2/6 \approx 0.042 + 0.167 = 0.21$ (with $h_1 = 0.5$ in x_1 , $h_2 = 1.0$ in x_2).

Error convergence with depth. Figure 5 quantifies the approximation error as a function of tree depth for both the one-dimensional ($d=1$) and two-dimensional ($d=2$) examples. In one dimension, each depth level halves the segment width h , reducing the maximum error $h^2/6$ by a factor of four—yielding a straight line of slope -2 on the log-log plot. In two dimensions with alternating axis splits, the error decreases by a factor of four every *two* depth levels (since only one axis is refined per level), producing a shallower slope of -1 on the log-log scale. At depth 8, the one-dimensional error (9.2×10^{-5}) and two-dimensional error (0.021) are both negligible relative to the function’s range.

4 Complexity Analysis

4.1 Training Time

Let n = training samples, d = features, T = number of nodes, $D = \text{max_depth}$. At each internal node with n_v samples:

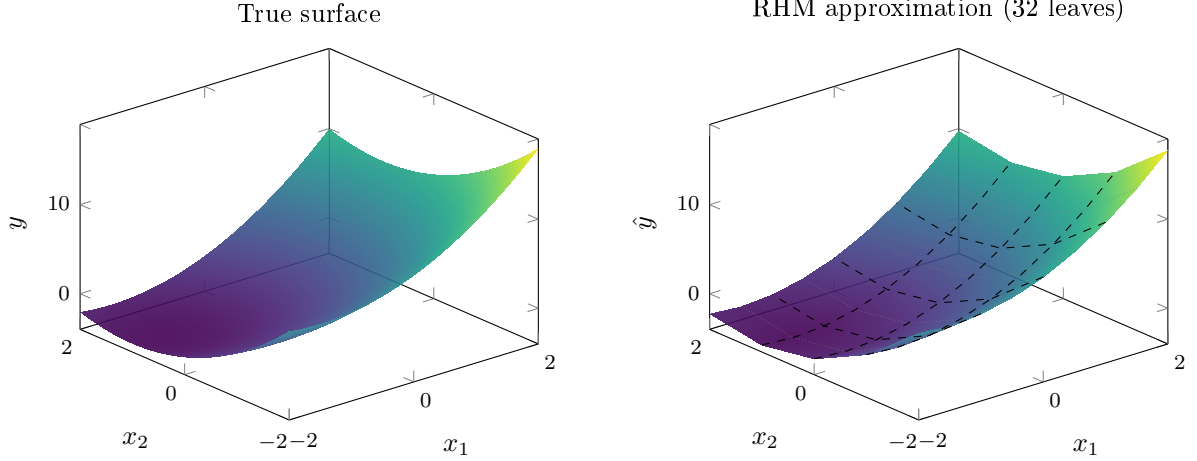


Figure 4: True surface $y = 3x_1 - 2x_2 + x_1^2 + x_2^2$ (left) and RHM piecewise planar approximation with 32 leaves on an 8×4 grid (right). Dashed lines mark the 4×4 major cell boundaries; dotted lines mark the finer x_1 subdivisions. The 32 Ridge planes are continuous across all boundaries and closely wrap the paraboloid with maximum error ≈ 0.21 .

- **Feature selection:** One Ridge fit costs $O(n_v d^2)$. Argmax over d coefficients costs $O(d)$. Total: $O(n_v d^2)$.
- **Split-point search:** 9 candidate splits, each requiring two Ridge fits. Cost: $O(9 \times 2 \times n_v d^2) = O(n_v d^2)$.

Because the left and right partitions together contain n_v samples and each sample appears in exactly one partition per level, the total work per depth level is $O(nd^2)$. Across D levels the total training cost is:

$$O(D \cdot n \cdot d^2)$$

This is linear in depth and quadratic in features, making RHM tractable for moderate d and small-to-medium D .

4.2 Inference Time

Each test sample traverses a path of at most $D+1$ nodes and then executes one Ridge prediction of $O(d)$. Per-sample inference cost is $O(D+d)$. For m test samples: $O(m(D+d))$.

4.3 Space Complexity

The tree contains at most $2^{D+1} - 1$ nodes. Each leaf stores a Ridge model with $O(d)$ parameters. Internal nodes store only a feature index and scalar threshold. Total space: $O(2^D \cdot d)$.

5 From RHM to ACGMT

5.1 Limitations of Top-1 Feature Selection

The preceding analysis shows that RHM converges correctly in idealised, low-dimensional settings where the splitting axis is unambiguous. In more realistic regression problems, however, the top-1 feature selection heuristic is brittle in several important ways.

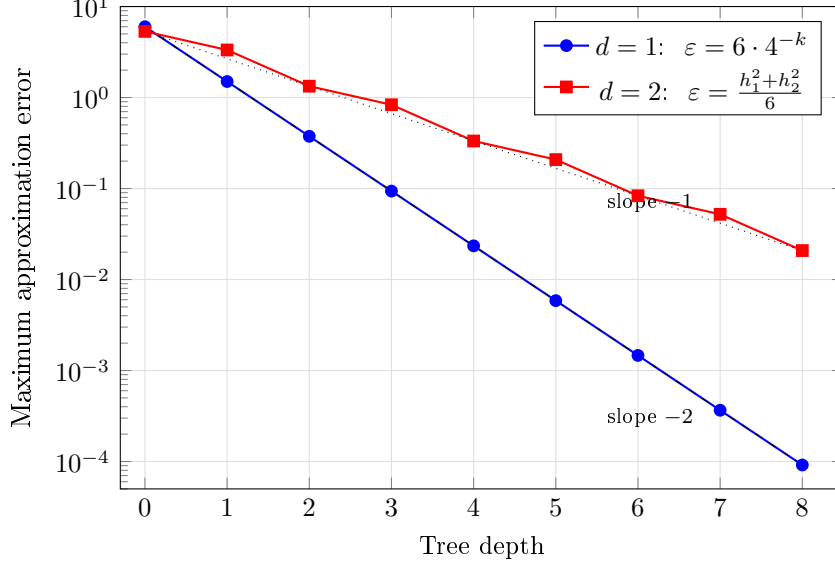


Figure 5: Maximum approximation error versus tree depth for $d = 1$ (blue circles) and $d = 2$ (red squares) on a semi-log scale. In one dimension, each depth level reduces the error by $4\times$ (slope -2 on a log-log scale, shown dashed). In two dimensions with alternating axis splits, the error halves per depth level on average (slope -1 , shown dotted), because each level refines only one axis. Both curves confirm the theoretical $O(h^2)$ convergence rate.

Sensitivity to feature scaling. Ridge coefficients are scale-dependent: a feature measured in large units will receive a proportionally small coefficient even if it is the most predictive axis. Without pre-standardization, the coefficient ranking can be dominated by the scale of the features rather than their predictive relevance.

Fragility under collinearity. When two predictive features are highly correlated, Ridge regression distributes their joint signal across both coefficients, each reduced in magnitude. The top-1 selection may commit to the feature that happened to receive a slightly larger share of this divided signal, discarding a potentially equally or more useful splitting axis.

Regime-dependent structure. In problems where the response is piecewise linear with distinct breakpoints that are not aligned with the single largest-coefficient feature, the top-1 heuristic will systematically fail to identify useful splits, producing large residual errors in the affected partitions.

These failure modes motivate a more flexible search strategy that uses coefficient magnitudes as a *screening tool* rather than a hard, irrevocable commitment. This is precisely the design goal of ACGMT.

5.2 Algorithm

Let (X, y) denote the data at a node, with $X \in \mathbb{R}^{n \times d}$ and $y \in \mathbb{R}^n$. ACGMT preserves the overall RHM structure but changes the split search from a single-axis decision to a screened multi-axis evaluation. Figure 6 illustrates the structural difference.

Step 1: coefficient-guided feature screening. If standardization is enabled, ACGMT centers and scales the node features column-wise before fitting Ridge. Let $\hat{\beta}$ denote the fitted coefficient vector. The algorithm ranks features by $|\hat{\beta}_j|$ and keeps the top- k candidates rather than only the single largest one.

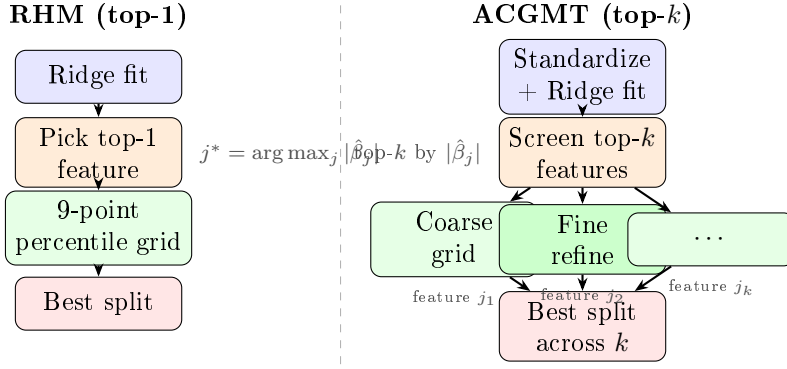


Figure 6: Conceptual comparison of split selection at a single node. RHM (left) commits to one feature and searches a fixed grid. ACGMT (right) standardizes features, screens the top- k by coefficient magnitude, searches each with optional coarse-to-fine refinement, and selects the best split across all k candidates.

Step 2: threshold search within screened features. For each retained feature, ACGMT first evaluates a coarse quantile grid. In our implementation, the coarse grid matches the 10th through 90th percentiles used by RHM. If adaptive refinement is enabled, the best coarse threshold is then refined by searching a smaller local window with a denser secondary grid.

Step 3: split selection. For every candidate threshold τ , the node is partitioned into left and right subsets and two child Ridge models are fit. The score is the weighted in-sample child MSE,

$$\text{MSE}(\tau) = \frac{|L| \cdot \text{MSE}_L + |R| \cdot \text{MSE}_R}{n}.$$

The feature-threshold pair with minimum score is selected, subject to the minimum node size constraint.

Step 4: recursion and pruning. Tree growth proceeds recursively until the depth limit or minimum sample constraint is reached. If pruning is enabled, the tree is first grown on a build split and then pruned bottom-up using a held-out validation subset. A subtree is collapsed into a single Ridge leaf whenever the leaf’s validation MSE is no worse than the subtree’s validation MSE on the validation samples that reach that node.

5.3 Variants Studied

The experiments study four ACGMT variants:

- **ACGMT (top1, no-std):** top-1 feature selection, no standardization, fixed grid, no pruning. This is the closest ACGMT analogue to RHM.
- **ACGMT (top3, std):** top-3 features, standardization, fixed grid, no pruning.
- **ACGMT (full):** top-3 features with standardization, adaptive threshold refinement, and validation-aware pruning.
- **Ablation variants:** versions that isolate top- k , standardization, adaptive grid search, and pruning separately.

5.4 Complexity

For fixed k , fixed coarse grid size, and fixed fine grid size, ACGMT has the same asymptotic training order as RHM up to a larger constant factor. Each internal node now fits one Ridge model for ranking and then evaluates up to k features across a bounded set of candidate thresholds. Thus, compared with RHM, ACGMT spends more computation per node but remains far cheaper than exhaustive search over all features and thresholds. For the settings used here, the practical fit-time overhead is modest relative to gradient boosting, while still preserving the single-tree structure.

6 Experimental Evaluation

6.1 RHM Baseline Evaluation

6.1.1 Synthetic Data Generation

To evaluate RHM in a controlled setting, we generate synthetic data with known linear, quadratic, and interaction structure. The target variable is:

$$y = 5x_1 - 3x_2 + \sum_{i=3}^{10} x_i + 0.5x_1^2 + 0.3x_2^2 + 2x_3x_4 + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.5)$$

where $x_i \sim \mathcal{N}(0, 1)$ i.i.d., $n = 5,000$ samples, $d = 10$ features. The quadratic and interaction terms are non-linear and cannot be captured by a global linear model, making this a suitable benchmark for assessing the benefit of local partitioning. Data are split 80/20 into training and test sets. All experiments use a fixed random seed (42) for reproducibility.

The baseline is a global Ridge regression ($\alpha = 1.0$) fitted on the full training set, and RHM is evaluated at $\text{max_depth} \in \{1, 2, 3, 4\}$ with $\text{min_samples_split} = 100$ throughout.

6.1.2 Results

Table 2 summarises the performance of the global Ridge baseline and RHM across depth levels.

Table 2: Test and training performance of RHM vs. Ridge baseline.

Model / Depth	Train MSE	Test MSE	Train R^2	Test R^2
Ridge (baseline)	4.9196	5.2845	0.8951	0.8849
RHM, depth=1	4.4381	4.8798	0.9053	0.8938
RHM, depth=2	4.2716	4.8746	0.9089	0.8939
RHM, depth=3	4.1593	4.9433	0.9113	0.8924
RHM, depth=4	4.0127	5.0727	0.9144	0.8896

Deeper trees capture the quadratic and interaction structure more accurately on training data, with train MSE decreasing monotonically from 4.92 (Ridge) to 4.01 (depth=4). On test data, RHM achieves its best performance at depth=2 (test MSE 4.87, R^2 0.894), after which test MSE increases slightly—consistent with mild overfitting as tree depth grows and leaf partitions shrink.

For $\text{max_depth} = 2$, four leaf models are fitted. Because x_1 has the largest Ridge coefficient at both the root and both depth-1 children (its linear coefficient of 5 dominates the other features), the tree splits on x_1 at all internal nodes—first near $x_1 \approx 0.0$, then near $x_1 \approx -0.8$ (left child) and $x_1 \approx 1.3$ (right child). This produces four vertical strips along the x_1 axis rather than a grid across two features. Exporting the leaf coefficients reveals how the x_1 coefficient adapts across strips: it is smallest in the far-left partition (where $x_1 < -0.8$ and the quadratic

$0.5x_1^2$ flattens the effective slope) and largest in the far-right partition (where $x_1 > 1.3$ and the quadratic steepens it). Coefficients on other features remain relatively stable across leaves, since the data-generating process couples non-linearity primarily to x_1 and x_2 .

6.2 ACGMT Stress Tests

6.2.1 Datasets

We evaluate on five synthetic regression problems designed to stress specific failure modes of coefficient-guided splitting:

- **Scaling:** features live on widely different numeric scales;
- **Correlated:** predictive features are paired with highly correlated surrogates;
- **Irrelevant:** a small number of relevant features are embedded among many irrelevant ones;
- **Interaction:** the response includes regime-dependent interaction effects;
- **Regime:** the response is piecewise linear with multiple breakpoints.

Each dataset contains 5,000 samples and is split 80/20 into train and test. We run 5 random seeds for every dataset-model pair and report means across seeds.

6.2.2 Baselines

We compare against:

- Ridge regression;
- CART regression trees;
- Random Forest;
- Gradient Boosting regression trees;
- the original RHM implementation from Section 3.

Random Forest uses single-threaded fitting in order to make fit-time comparisons less misleading. We emphasize fit time rather than prediction time when discussing efficiency, because the current RHM implementation uses slower row-wise prediction while ACGMT uses vectorized prediction.

6.2.3 Protocol Details

All non-pruned ACGMT variants train on the full training set. The pruned ACGMT (`full`) variant uses an 80/20 build/validation split within the training data, so its tree is grown on 80% of the nominal training set and pruned using the held-out 20%. The pruning ablation compares pruned and unpruned models built on the same 80% subset, so only the pruning step differs.

6.2.4 Main Comparison

Table 3 reports test MSE and fit time. The headline result is that adaptive coefficient-guided search substantially improves over RHM on datasets where top-1 feature commitment fails, especially the interaction and regime settings. RHM attains test MSE 4.746 on the interaction dataset and 2.791 on the regime dataset, while ACGMT (`top3`, `std`) reduces these to 0.385 and 0.352 respectively. The full pruned variant further improves the correlated and regime datasets, reaching 0.253 and 0.325.

Table 3: Main comparison across datasets. Test MSE is lower-is-better. Fit time is mean wall-clock training time in seconds.

Dataset	Ridge	CART	RF	GBT	RHM	ACGMT top1	ACGMT top3+std	ACGMT full
<i>Test MSE</i>								
correlated	0.711	2.784	2.068	0.335	0.277	0.277	0.259	0.253
interaction	5.102	1.947	1.387	0.352	4.746	4.746	0.385	0.389
irrelevant	1.810	7.424	5.002	0.453	0.316	0.316	0.303	0.310
regime	2.758	0.950	0.820	0.274	2.791	2.791	0.352	0.325
scaling	6.492	10.649	6.833	0.615	3.020	3.020	2.963	2.998
<i>Fit time (s)</i>								
correlated	0.000	0.004	0.278	0.438	0.169	0.061	0.171	0.252
interaction	0.000	0.004	0.282	0.446	0.128	0.054	0.182	0.281
irrelevant	0.001	0.016	1.011	1.692	0.223	0.082	0.227	0.349
regime	0.000	0.005	0.284	0.450	0.139	0.052	0.169	0.244
scaling	0.000	0.004	0.283	0.447	0.195	0.067	0.167	0.251

Figure 7 presents the principal test-MSE comparisons graphically (CART and the top-1 ACGMT variant, which closely tracks RHM, are omitted for clarity). Three comparisons are particularly informative. First, ACGMT (top1, no-std) closely tracks RHM on every dataset, confirming that the gain does not come merely from reimplementing. Second, ACGMT (top3, std) is the strongest non-pruned variant and delivers the largest improvements over RHM. Third, ACGMT (full) does not uniformly dominate top3+std; pruning and adaptive refinement are useful but context-dependent. This is an important result: the strongest contribution of the ACGMT line of work is not “adding every enhancement,” but establishing which enhancements actually matter.

Compared with ensemble baselines, ACGMT occupies a middle ground. Gradient boosting remains best on the most non-linear scaling and interaction problems, but ACGMT is competitive on correlated and irrelevant-feature settings and remains a single interpretable tree rather than an ensemble of many trees.

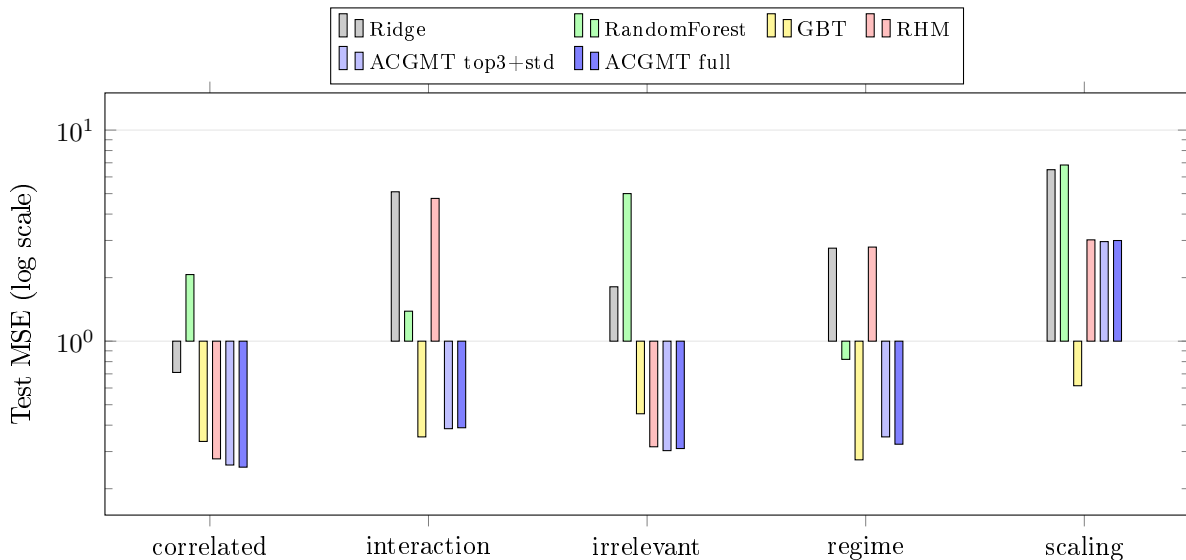


Figure 7: Test MSE comparison across datasets (log scale). RHM fails on the interaction and regime datasets (tall red bars), while ACGMT variants (blue) reduce error by an order of magnitude. GBT (yellow) is strongest on scaling and interaction. ACGMT beats GBT on correlated and irrelevant data.

6.2.5 Ablation Study

Table 4 summarizes the ablations.

Table 4: Ablation study: test MSE across datasets.

Dataset	Top- k				Standardize		Grid		Pruning	
	top-1	top-2	top-3	top-5	no-std	with-std	adaptive	fixed	no-prune	with-prune
correlated	0.265	0.259	0.259	0.259	0.259	0.259	0.259	0.259	0.263	0.253
interaction	0.528	0.385	0.385	0.385	0.385	0.385	0.374	0.385	0.388	0.389
irrelevant	0.336	0.310	0.303	0.304	0.303	0.303	0.304	0.303	0.316	0.310
regime	2.441	0.352	0.352	0.353	0.352	0.352	0.337	0.352	0.331	0.325
scaling	5.122	3.869	2.963	1.719	1.719	2.963	3.080	2.963	3.002	2.998

Figures 8 and 9 present the ablation results graphically. The ablations support four conclusions.

Top- k screening is the dominant enhancement. Moving from top-1 to top-2 or top-3 produces the largest single gains. On the regime dataset, test MSE drops from 2.441 to 0.352; on the interaction dataset it drops from 0.528 to 0.385; on the scaling dataset it drops from 5.122 to 2.963 by top-3 and to 1.719 by top-5. The main weakness of RHM is therefore not merely threshold search, but the early commitment to a single splitting feature.

Standardization is not uniformly helpful. We expected node-wise standardization to stabilize coefficient ranking, but in these experiments it is neutral on four datasets and harmful on the scaling stress test. This is a useful negative result. Once top- k screening is already exploring multiple features, naive standardization can alter the ranking in ways that do not improve split quality.

Adaptive threshold search helps when breakpoint precision matters. Adaptive refinement improves the regime dataset from 0.352 to 0.337 and the interaction dataset from 0.385 to 0.374, while remaining nearly neutral elsewhere. Its effect is therefore smaller than top- k screening but still meaningful for problems with sharper local transitions.

Pruning is a small but generally positive refinement. In the clean pruning ablation, pruning improves correlated, irrelevant, regime, and scaling data, and is effectively neutral on interaction. The gains are modest, which is exactly what one would expect from a bottom-up regularization step rather than a fundamentally different split-selection mechanism.

6.2.6 Complexity-Contextualized Comparison: Performance-Equivalent Boosting Budgets

The main comparison in Table 3 evaluates ACGMT against gradient boosting regression (GBR) with 100 depth-4 estimators. This is a useful practical comparison, but it is not complexity-matched: a single depth-4 model tree is evaluated against an additive ensemble of one hundred boosted trees, each fitted to residuals of the previous. To contextualize this gap, we conduct a complexity sweep over GBR ensemble sizes ($n_{\text{estimators}} \in \{1, 2, 4, 8, 16, 32, 64, 100\}$, all at depth 4) and identify, for each dataset, the smallest ensemble size at which GBR first matches or exceeds the test R^2 of ACGMT (depth 4, full configuration). We refer to this quantity as the *empirical performance-equivalent boosting budget*. This analysis does not constitute a proof of equivalent model capacity—the representational structures of a single linear-model tree and a

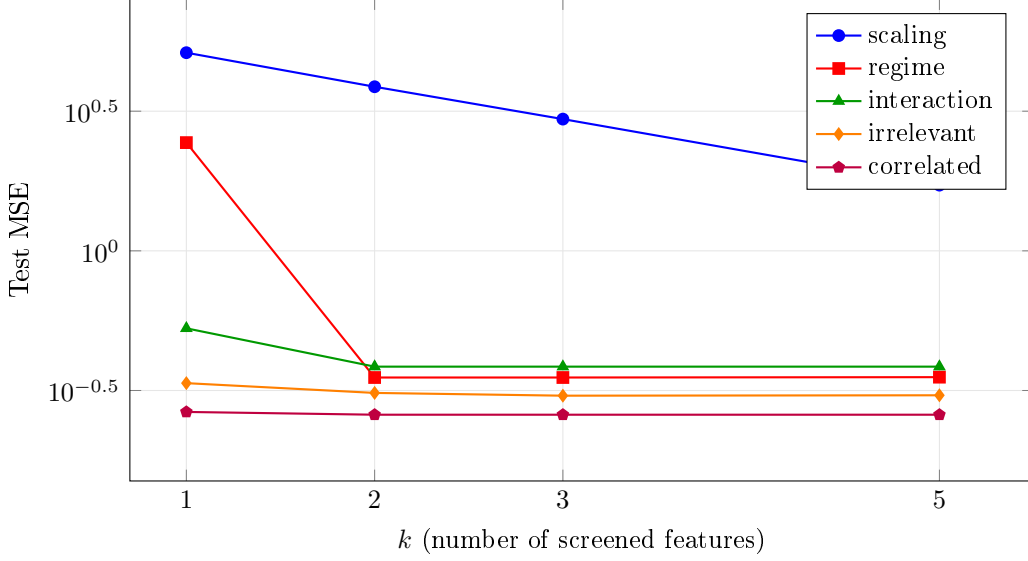


Figure 8: Effect of top- k feature screening on test MSE (log scale). The largest gains occur from $k = 1$ to $k = 2$. The regime dataset drops from 2.44 to 0.35—a $7\times$ reduction—when the algorithm considers a second feature. Scaling continues to improve through $k = 5$.

boosted ensemble remain fundamentally distinct—but it provides an empirically grounded basis for contextualizing the single-tree results within an ensemble complexity scale.

The results reveal a pronounced dependence on problem structure. On the two synthetic datasets designed to exhibit piecewise-linear or axis-aligned structure—the correlated-features and irrelevant-features benchmarks—a single ACGMT tree achieves R^2 of 0.983 and 0.992 respectively, and GBR does not match either value within the tested sweep of 100 estimators (GBR at 100 estimators reaches 0.978 and 0.988). These results indicate that the inductive bias of ACGMT, which partitions the input space into locally linear regions guided by regression coefficient structure, is particularly well-suited to problems where the generative process is itself approximately piecewise linear.

On three moderately nonlinear benchmarks—the interaction, regime, and friedman1 datasets—ACGMT achieves R^2 values of 0.953, 0.914, and 0.850 respectively. GBR requires approximately 32 estimators to match performance on the regime and friedman1 datasets, and between 32 and 64 estimators on the interaction dataset. That GBR at 100 estimators ultimately reaches higher values (0.957, 0.928, and 0.937) confirms that boosting retains a peak-accuracy advantage on these problems, but the performance-equivalent boosting budget of roughly 32 estimators indicates that ACGMT’s single-tree solution is competitive with a meaningfully sized ensemble.

On california housing, concrete compressive strength, and airfoil self-noise, GBR reaches ACGMT-level performance at approximately 16 estimators, and larger ensembles ultimately achieve higher peak accuracy (R^2 of 0.811, 0.923, and 0.884 against ACGMT’s 0.696, 0.838, and 0.702). Diabetes behaves differently: ACGMT (0.383) remains slightly above GBR(100) (0.352), while pruning collapses the tree to depth 1 across all tested depths, suggesting limited recoverable piecewise-linear structure under the current configuration.

The two remaining datasets represent the clearest limits of the ACGMT approach. On the yacht hydrodynamics dataset, ACGMT achieves an R^2 of only 0.577, and GBR at 4 estimators already approaches this value (approximately 0.55), with GBR at 100 estimators reaching 0.998. Pruning again collapses ACGMT to a depth-1 tree, consistent with a strongly nonlinear response surface that piecewise-linear partitioning cannot efficiently represent. On the scaling benchmark, ACGMT reaches 0.854 (improving through depth 5 to 0.868), while GBR requires approximately 32 estimators to match and ultimately achieves 0.970 at 100 estimators. On such problems,

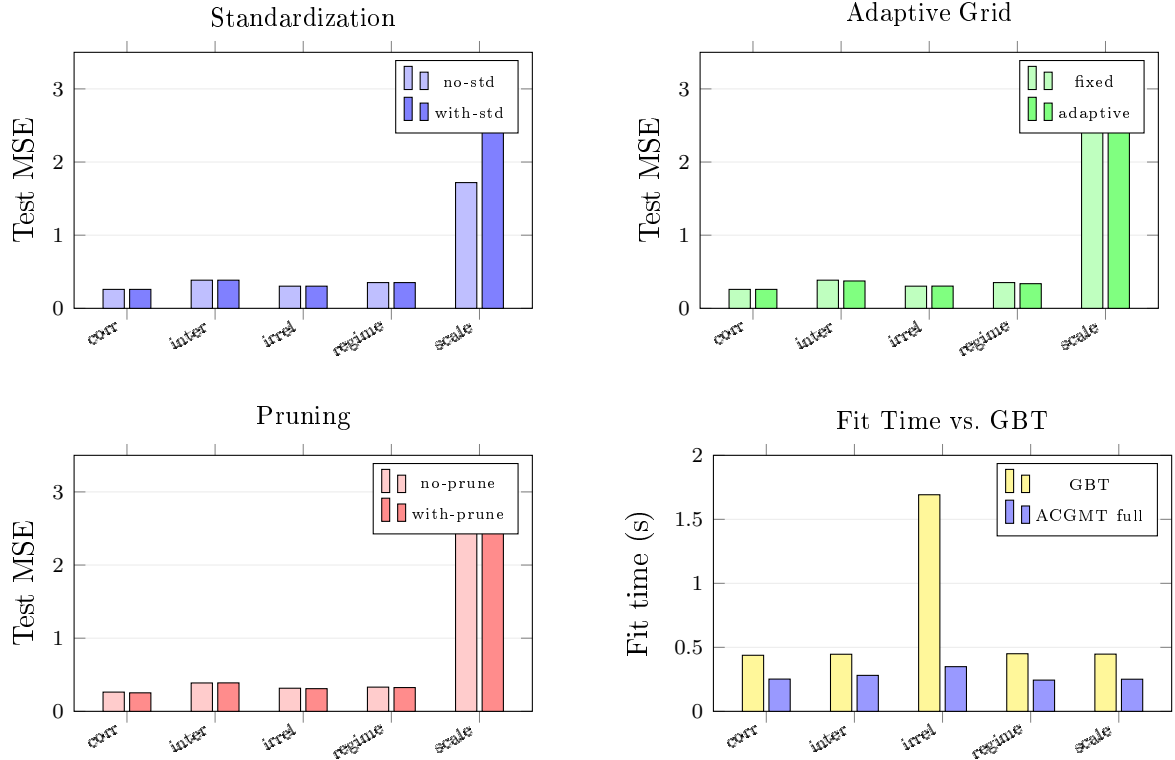


Figure 9: Ablation panels. Top-left: standardization is neutral on 4/5 datasets and harmful on scaling. Top-right: adaptive grid refinement helps regime (0.352→0.337) and interaction (0.385→0.374). Bottom-left: pruning gives small improvements on 4/5 datasets. Bottom-right: ACGMT full fits faster than GBT on all datasets.

boosting with many estimators provides substantially higher peak accuracy, and ACGMT should not be expected to close that gap.

The complexity sweep also reveals a structural advantage in prediction cost. At inference time, ACGMT traverses a single tree and evaluates one local linear model (a dot product of d coefficients), whereas GBR must traverse and sum the outputs of all $n_{\text{estimators}}$ trees. Table ?? reports mean prediction time over five seeds. On california housing ($m = 4,128$ test samples), ACGMT (depth 4) predicts in 0.39 ms compared to 4.45 ms for GBR(100)—an $11\times$ speedup. On friedman1 ($m = 1,000$), the ratio is $8\times$ (0.18 ms vs 1.46 ms). Even at the performance-equivalent ensemble size of GBR(32), ACGMT (depth 4) predicts 2–4 $\times$ faster. This advantage is a direct consequence of the single-tree architecture: prediction cost scales with tree depth rather than ensemble size, making ACGMT particularly attractive in deployment settings where prediction latency is constrained.

Table 5: Mean prediction time (ms) over 5 seeds. ACGMT predicts faster than GBR at all ensemble sizes because it traverses a single tree and evaluates one linear model per sample.

Dataset	m	ACGMT (d=4)	GBR(32)	GBR(100)	Speedup
california	4,128	0.39	1.75	4.45	$11\times$
friedman1	1,000	0.18	0.50	1.46	$8\times$
interaction	1,000	0.32	0.39	1.17	$4\times$
concrete	206	0.07	0.10	0.38	$5\times$
airfoil	301	0.14	0.12	0.59	$4\times$

Taken together, these results provide a more contextualized comparison between ACGMT and gradient boosting. ACGMT is not uniformly competitive with GBR across all problem classes, nor should it be interpreted as a drop-in replacement for high-capacity ensembles. However, on datasets with structured or approximately piecewise-linear relationships, a single ACGMT tree attains the test performance that GBR reaches only after a nontrivial number of boosting stages—while predicting several times faster. This suggests that, in such regimes, ACGMT can offer a favorable tradeoff between predictive performance, prediction latency, and model inspectability.

7 Discussion

7.1 RHM: Advantages and Limitations

The RHM experiments confirm that coefficient-guided model trees can outperform a global Ridge baseline on non-linear data with quadratic and interaction structure. The key advantages of the RHM design are threefold.

Linear interpretability within regions. Unlike CART leaves (scalar constants), RHM leaves provide a full linear equation, enabling covariate-level attribution within each region.

Efficiency of coefficient-guided splitting. By using Ridge coefficients to select the splitting feature, RHM avoids the $O(d)$ -factor cost of CART’s exhaustive feature search while still using data-driven evidence for the choice.

Regularised leaf models. Ridge regression at the leaves prevents overfitting of individual leaf models, particularly important when deep trees produce small partitions.

The limitations of RHM are equally clear. The feature with the largest Ridge coefficient in the current partition is not guaranteed to be the optimal splitting feature according to the MSE criterion. The fixed percentile grid of 9 candidate split points may miss optimal thresholds that lie between percentiles. Hard binary routing creates discontinuities at region boundaries. And the algorithm does not include post-hoc pruning or cross-validation for depth selection, requiring external depth tuning.

These limitations motivate the progression from RHM to ACGMT described in Section 5.

7.2 ACGMT: What the Experiments Imply

The ACGMT experiments suggest that coefficient-guided model trees are viable, but only when the coefficient signal is used as a *screening tool* rather than a hard one-feature decision rule. This is the central lesson of the ACGMT study. RHM’s top-1 heuristic fails badly on interaction and regime data; once the search is expanded to a small set of plausible features, the method becomes far more robust.

The results also clarify what ACGMT is *not*. It is not a universal replacement for strong tree ensembles. Gradient boosting remains superior on strongly nonlinear problems (yacht, scaling) and achieves higher peak accuracy on most real-world datasets when given a large ensemble budget. Nor does every proposed enhancement help equally: top- k screening is the clear driver of improvement, adaptive threshold refinement provides targeted gains, pruning gives small regularization benefits, and standardization is at best mixed.

The complexity sweep against gradient boosting (Section 6) adds an important nuance. The analysis does not imply that ACGMT and GBR possess equivalent representational capacity; rather, it establishes empirically that on several structured regression problems, the effective accuracy obtained from a single interpretable model tree falls within the range that GBR spans

during its early-to-middle boosting iterations, before further estimators yield diminishing returns. On problems with approximately piecewise-linear structure, ACGMT matches the performance that GBR requires roughly 32 or more estimators to attain. On strongly nonlinear targets, GBR overtakes ACGMT with as few as 4–16 estimators. This pattern is consistent with the view that coefficient-guided model trees are most valuable when the underlying relationship is structured enough to be well-approximated by a small number of local linear models.

These observations lead to a more precise methodological claim. The value of coefficient-guided model trees lies not in a single heuristic, but in a family of constrained search strategies that trade exhaustive split search for a guided restricted search. ACGMT represents one such strategy, and the hierarchy of importance established by ablation—top- k first, adaptive grid and pruning conditional, standardization cautious—is itself a key finding.

There are clear limitations to the present study. First, the comparison does not include an external exhaustive model-tree implementation (e.g., M5’); the focus is on the progression from RHM to ACGMT and on standard tabular baselines. Second, the present RHM implementation uses row-wise prediction, making fit time the cleaner efficiency metric compared to ACGMT’s vectorized inference. Third, on datasets with very few samples (diabetes, yacht), pruning collapses the tree to depth 1, preventing meaningful evaluation of the tree-building enhancements.

7.3 Future Directions

- (a) Replacing the coefficient-based heuristic with a fast approximate feature search (e.g., random subset of features) or hybrid strategy for better split quality at moderate cost.
- (b) Implementing M5’-style smoothing across leaf boundaries to improve continuity of the piecewise linear predictor.
- (c) Extending to multi-output regression by replacing Ridge with multi-output Ridge at each node and leaf.
- (d) Integrating cross-validated depth selection or cost-complexity pruning into the RHM/ACGMT framework.
- (e) Investigating the theoretical regime in which top- k screening achieves the same convergence guarantees as exhaustive feature search.
- (f) Comparing against external model-tree implementations (e.g., M5’, PILOT, PINE) on standardized benchmark suites.

8 Conclusion

This paper presented a unified development of coefficient-guided model trees for piecewise linear regression, tracing the progression from a minimal baseline to a robust adaptive variant.

The Recursive Hybrid Model (RHM) established that a Ridge-coefficient-guided feature selection heuristic—selecting the splitting axis as the feature with the largest absolute coefficient magnitude—produces a clean, analysable algorithm with training complexity $O(D \cdot n \cdot d^2)$, inference complexity $O(m(D + d))$, and the classical $O(h^2)$ piecewise linear convergence rate in idealised settings. Empirical evaluation confirmed that RHM consistently outperforms a global Ridge baseline on synthetic non-linear data, with gains increasing with depth before diminishing returns set in.

The preceding analysis also identified the primary weakness of the top-1 heuristic: early commitment to a single splitting feature causes failures on data with scaling distortions, correlated predictors, and regime-dependent structure. The Adaptive Coefficient-Guided Model Tree (ACGMT) addressed this by promoting the coefficient signal from a hard decision rule

to a screening device: the algorithm ranks features by coefficient magnitude, retains the top- k candidates, and evaluates each for the best split. Ablation experiments on five synthetic stress tests showed that this single change is the dominant improvement—test MSE dropped by an order of magnitude on interaction and regime datasets when moving from top-1 to top-2 or top-3. Adaptive threshold refinement and validation-aware pruning provided supplementary, context-dependent gains; standardization was neutral to harmful.

A complexity sweep over gradient boosting ensemble sizes provides further context. On datasets with approximately piecewise-linear generative structure, a single ACGMT tree attains the predictive performance that gradient boosting reaches only after accumulating roughly 32 or more estimators, while remaining a fully inspectable model whose local linear components can be directly examined. On strongly nonlinear problems, boosting with many estimators achieves substantially higher peak accuracy, and practitioners should regard these cases as outside the method’s primary domain of applicability.

The broader implication is that coefficient-guided model trees should be understood as *adaptive search procedures*, not one-shot heuristics. The resulting ACGMT remains a single interpretable tree and significantly improves over the original coefficient-guided baseline on the problem classes where the baseline fails. The choice between ACGMT and a boosted ensemble should be guided by the degree to which interpretability and local linearity are operationally valued: on structured problems where the two approaches reach comparable accuracy, ACGMT provides fully inspectable local linear models at the cost of reduced peak performance on targets that exhibit complex global nonlinearity.

References

- Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- Cerlymarco. (2021). linear-tree: A python library to build Model Trees with Linear Models at the leaves. GitHub. <https://github.com/cerlymarco/linear-tree>
- Friedman, J. H. (1979). A Tree-Structured Approach to Nonparametric Multiple Regression. In T. Gasser & M. Rosenblatt (Eds.), *Smoothing Techniques for Curve Estimation*, Lecture Notes in Mathematics, vol. 757. Springer.
- Friedman, J. H. (1991). Multivariate Adaptive Regression Splines. *The Annals of Statistics*, 19(1), 1–67.
- Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, 29(5), 1189–1232.
- Hansen, B. E. (2000). Sample Splitting and Threshold Estimation. *Econometrica*, 68(3), 575–603.
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67.
- Li, Z., et al. (2024). PINE: Efficient yet Effective Piecewise Linear Trees. *SC24 Supercomputing Conference Proceedings*. https://sc24.supercomputing.org/proceedings/poster/poster_files/post258s2-file3.pdf
- Quinlan, J. R. (1992). Learning with Continuous Classes. *Proceedings of the 5th Australian Joint Conference on Artificial Intelligence*, pp. 343–348.

- Raymaekers, J., Rousseeuw, P. J., Verdonck, T., & Yao, R. (2024). Fast Linear Model Trees by PILOT. *Machine Learning*. <https://doi.org/10.1007/s10994-024-06590-3>
- Wang, Y., & Witten, I. H. (1997). Induction of Model Trees for Predicting Continuous Classes. *Proceedings of the European Conference on Machine Learning*, pp. 128–137.